Санкт-Петербургский государственный университет
Факультет прикладной математики — процессов управления
ООП СВ.5005.2022 Прикладная математика, фундаментальная информатика и программирование

Шаго Павел Евгеньевич

Математическое моделирование и реализация планировщика процессов для гетерогенных процессорных архитектур

Бакалаврская работа

Научный руководитель:
кандидат физ.-мат. наук, доцент
Корхов В. В.

Рецензент:
эксперт по архитектуре и сетевому стеку,
ООО НИЦ АЭРОДИСК, Анохин Ю. В.

Санкт-Петербург
2026

Введение

Постановка задачи

- Стандартный планировщик в Linux является частью ядра и не может быть модифицирован без его модификации и пересборки
- This work investigates reimplementation of EEVDF via sched_ext to study how different implementation approach can affect scheduling performance
  Данная работа исследует варианты
- To evaluate the implementation, standard workload benchmarks are used such as hackbench, sysbench, and eBPF-based scheduling latency tracing
- Then the sched_ext variant is compared against the built-in kernel EEVDF to identify trade-offs introduced by eBPF dispatch

sched_ext (scheduler extensions) is a new Linux kernel infrastructure that enabled implementing CPU scheduling policies as eBPF programs attached to kernel scheduling hooks at runtime

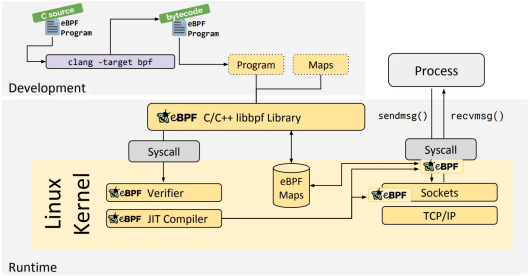

Figure 1. eBPF in the network subsystem

DSQs (dispatch queues):

- SCX_DSQ_LOCAL – per-core fast path
- SCX_DSQ_GLOBAL – shared queue

Scheduler cycle:

- select_cpu – pick idle cpu core
- enqueue – place task in global DSQ
- dispatch – move to local DSQ
- running / stopping – based on vtime

Earliest Eligible Virtual Deadline First Model

The ideal reference system is such system where every active client i continuously receives share $w_i/W(t)$ of the CPU. EEVDF tracks this entitlement through system virtual time $V(t)$,

$$W(t) = \sum_{j \in A(t)} w_j, \qquad V(t) = \int_{t_0}^{t} \frac{d\tau}{W(\tau)}.$$

When client i submits a request of size r, the algorithm assigns a *virtual eligible time* and a *virtual deadline*,

$$v_e = V(t_0^i) + \frac{s_i(t_0^i, t)}{w_i}, \qquad v_d = v_e + \frac{r}{w_i},$$

where $s_i(t_0^i, t)$ is the service received since t_0^i.

Earliest Eligible Virtual Deadline First Model

A request is runnable only once $v_e \leq V(t)$. Among all eligible requests, EEVDF dispatches the one with the smallest virtual deadline v_d.

A client behind its proportional share carries a smaller v_e and becomes eligible sooner. Larger weights compress r/w_i, so heavier tasks earn earlier deadlines for the same request size.

The lag of client i measures the signed deviation between ideal model entitlement and actual service received. EEVDF guarantees [1] $|\text{lag}_i(t)| < q$ in steady state, the optimal bound for any discrete proportional-share scheduler with quantum q.

Mapping to sched_ext

Client	$\rightarrow$	task_struct
w_i	$\rightarrow$	p->scx.weight
v_i	$\rightarrow$	scx.dsq_vtime
$V(t)$	$\rightarrow$	vtime_now
r	$\rightarrow$	SCX_SLICE_DFL (Δ)

S – integer scaler

enqueue:

$$VE_i = \max(v_i,\ V_{\text{now}} - \Delta), \quad v_i \leftarrow VE_i$$

$$VD_i = VE_i + \frac{\Delta \cdot S}{w_i} \rightarrow \text{inserted by } VD_i$$

running:

$$V_{\text{now}} \leftarrow \max(V_{\text{now}},\ VE_i)$$

stopping:

$$v_i \leftarrow v_i + \frac{\text{consumed} \cdot S}{w_i}$$

Environment

System

Kernel:
Linux 7.0-rc4

CPU:
Intel Core
Ultra 7 265K

Comparison

Built-in EEVDF
vs
sched_ext EEVDF

Benchmarks

- **hackbench** – 1000 tasks
  Groups of processes exchange messages intensively, simulating high system load. Reflects context-switching and task coordination efficiency.
- **sysbench** – default parameters
  CPU, memory, and disk I/O throughput. Simulates OLTP database operations under realistic conditions.
- **sched-latency** – custom eBPF
  Attaches to tracepoints and records p99 schedule latency.

Metrics

hackbench
Total time (s)

sysbench
$Events/s$

sched-latency
p99 lat (μs)

Results

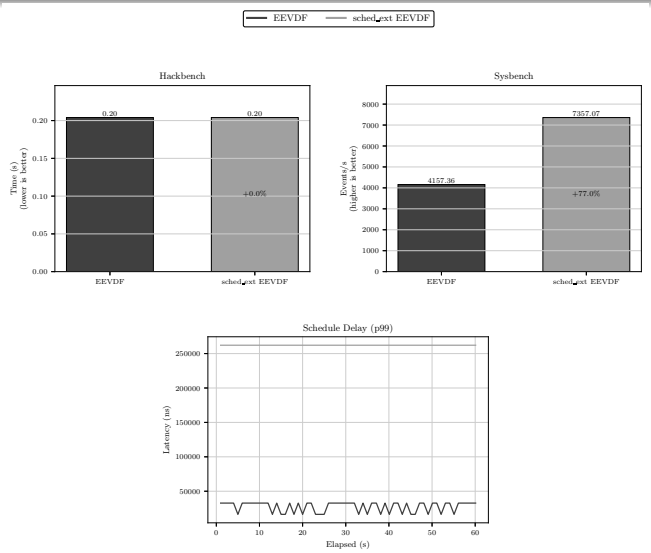

Figure 2. Test results for builtin EEVDF scheduler and implemented sched_ext EEVDF

Analysis

- **Hackbench:** 0.20 s for both schedulers (+0.0%)

→ eBPF dispatch overhead is negligible for intensive, scheduler demanding workloads

- **Sysbench:** 4157 → 7357 events/s

→ +77% – may offer advantages for sustained computational workloads

- **Scheduling latency (p99):** 25 μs → 250 μs

→ 10× increase attributable to eBPF dispatch overhead and contention on the single shared DSQ

Proposed sched_ext EEVDF achieves in-kernel EEVDF performance parity while decoupling scheduling policy from the kernel and providing greater flexibility

Conclusion

- EEVDF scheduling algorithm is faithfully reproducible outside the kernel via sched_ext and eBPF
- Policy changes take effect without recompilation or reboot, enabling rapid iteration
- eBPF dispatch overhead is significant, overall performance parity with in-kernel EEVDF is achievable
- Results lay the groundwork for future research targeting heterogeneous CPU platforms

References & Q/A

[1] Stoica I., Abdel-Wahab H. Earliest eligible virtual deadline first. PhD thesis. Old Dominion University, 1995

[2] An EEVDF CPU scheduler for Linux (LWN)

[3] The extensible scheduler class (LWN)

Proof that $|\text{lag}_i| < \Delta$ is preserved

- In the continuous model, a sleeping task accumulates negative lag $-\tau$ for virtual time τ it was absent
- Unclamped, it would re-enter with an ancient v_i and claim up to τ units of ”catch-up” service
- The clamp enforces $VE_i \geq V_{\text{now}} - \Delta$, so at most one quantum Δ of credit is carried forward
- $\text{lag}_i = (\text{ideal service}) - (\text{actual service}) \leq \Delta$ at the moment of re-enqueue, which is exactly the bound $|\text{lag}_i| < q$ with $q = \Delta$

The clamp is not a deviation from EEVDF – it *enforces* the lag bound that the continuous model satisfies by construction

Extra

Critiques: (a) lazy advancement breaks the $V(t)$ definition; (b) total_weight is never used in the scheduling formulas.

(a) Lazy advancement:

- EEVDF requires $V(t)$ to be monotonically non-decreasing and to track the frontier of virtual-time consumption
- vtime_now $\leftarrow \max(\text{vtime\_now}, v_i)$ in running() is exactly such a monotone frontier
- Dispatch ordering depends only on VD_i comparisons, not on the absolute value of V_{now} so ordering is preserved

(b) total_weight is used – in set_weight():

- When a task's weight changes, the lag must be rescaled: $\text{lag} \cdot W_{\text{old}}/W_{\text{new}}$
- The code adjusts vtime_now by exactly this proportional correction, preserving relative virtual positions
- $W(t)$ is not needed in the per-enqueue formulas because lazy advancement already subsumes $1/W(t)$ integration

Extra

Critique: S appears without justification; if too small, $\Delta \cdot S/w_i$ truncates to zero.

$S = \text{SCALE} = 1000$ (source: scx_eevdf.bpf.c:20)

Precision analysis (sched_ext weight range: $1 \leq w \leq 10000$; $\Delta = \text{SCX\_SLICE\_DFL} = 5\,\text{ms}$):

- $w = 1$: $\Delta \cdot S/w = 5 \times 10^9$
- $w = 10000$: $\Delta \cdot S/w = 500\,000$
- Ratio: $10\,000\times$ – exactly proportional

Non-zero for all $w \leq 5 \times 10^9$

Proportional-share property holds: heavier tasks get larger w_i, smaller $\Delta \cdot S/w_i$, earlier virtual deadline

S cancels in all fairness ratios; only the ratio $\Delta \cdot S/w_i$ matters, and it is exact integer arithmetic

Hypothesis – slice length difference:

- Kernel EEVDF dynamically shortens the time slice under contention: $q = q_{\text{min}} \cdot \log_2 n$ (where n is the number of runnable tasks)
- sched_ext uses a fixed $\Delta = \text{SCX\_SLICE\_DFL} = 5\,\text{ms}$ regardless of task count
- Sysbench is a single-threaded CPU-bound workload; with few tasks, the kernel EEVDF still applies its default shorter quanta and context-switches more often
- Fewer preemptions $\Rightarrow$ lower context-switch overhead $\Rightarrow$ higher throughput – consistent with the hackbench result (identical: 0.20 s) where many tasks eliminate the slice-length advantage

The schedulers implement the same EEVDF *policy* and the throughput gap traces to a *parameter* difference, not an algorithmic one